



DOSSIER DE LA PLATAFORMA · 2026

DistiNode

Diseña sistemas distribuidos en equipo
y míralos funcionar en vivo.



 / Demo Se guarda solo en este navegador

Proyecto Guía T ?

Componentes

Arrástralos al lienzo.

Tráfico

-  **Cliente**
-  **CDN**
-  **API Gateway**
-  **Balanceador**

Cómputo

-  **Servidor**
-  **Worker**

Datos

-  **Caché**
-  **Cola de mensajes**
-  **Base de datos**

Plantillas...

Une la salida ● derecha de una estación con la entrada izquierda de otra.
Espacio ▶ / # - Supr borra - Guía

 **Tráfico** 150 pet/s

completadas 144/s

latencia p95 122 ms

errores 0 %

en segundo plano 60/s · 0 en cola

Caos Sobrecarga tumba

ReactFlow

Una pizarra que funciona

DistiNode es una plataforma web educativa, en español, para **diseñar arquitecturas distribuidas como un mapa de metro**, ponerlas en marcha y ver en tiempo real dónde se atasca el tráfico, qué pasa cuando algo se cae y cómo se recupera. Varias personas editan el mismo diagrama a la vez.

9**componentes**

del cliente a la base de datos

14**plantillas**

patrones y sistemas reales

6**tipos de fallo**

caídas, lentitud, cortes, latencia, caos, sobrecarga

<100 ms**sincronización**

entre los navegadores de una sala

60 fps**animación**

trenes que son peticiones

**Diseña**

Arrastra estaciones al lienzo y únelas con líneas. Ajusta capacidad, colas, aciertos, límites o algoritmos en un panel lateral.

**Simula**

Pulsa ▶: los trenes recorren las líneas. Cada estación cambia de color según su carga y aparecen métricas como en un panel de producción.

**Rompe y aprende**

Tumba servidores, corta conexiones, añade latencia o activa el caos. Observa reintentos, colas y balanceadores reaccionando.

Para quién

- **Estudiantes** que quieren entender balanceo, cachés, colas, réplicas y tolerancia a fallos tocando, no solo leyendo.
- **Profesores** que necesitan una demostración en vivo en clase o un laboratorio guiado con retos.
- **Equipos** que esbozan una arquitectura y quieren ver si aguanta antes de construirla.

Contenido

01	Qué es DistiNode	2
02	Identidad visual	3
03	Cómo se usa	4
04	Los 9 componentes	5
05	Simulación, métricas y fallos	6
06	14 plantillas	7
07	Equipo, proyectos y móvil	9
08	Despliegue y tecnología	10

Conceptos que se aprenden haciendo

Escalado horizontal

Balanceo de carga

Chequeos de salud

Cachés y CDN

Colas y asincronía

Limitación de ritmo

Reintentos y backoff

Tormentas de reintentos

Tiempos de espera

Fan-out y latencia de cola

Particiones de red

Latencia entre regiones

Puntos únicos de fallo

Entrega al menos una vez

Teoría de colas (**p**)

Percentil 95

Identidad visual

La identidad parte de los colores institucionales de la CUJAE —verde y gris— y de un logo propio: cuatro cubos isométricos (nodos) dentro de un hexágono abierto (la red). El lenguaje de la interfaz es el de un **mapa de metro**: estaciones, líneas y trenes.



- «Disti» en verde, «Node» en tinta: una sola palabra, sin espacio.
- Área de respeto: la altura de un cubo alrededor del símbolo.
- El símbolo solo se usa en verde/gris; nunca rotado ni con sombras.

Paleta

#00704F Verde CUJAE Acción principal, líneas, estado sano	#005A3F Verde profundo Hover, énfasis, texto sobre claro	#A6A6A6 Gris acero Caras del logo, trenes, estado caído
#14211C Tinta Texto y contorno de estaciones	#E3F1EB Verde suave Fondos de selección y avisos	#F3F6F4 Fondo Lienzo con rejilla de puntos

Estados (siempre con texto o icono, no solo color)

- Sano Cargado · ámbar #B86E00 Saturado · rojo #C2362F
- Caído · gris discontinuo

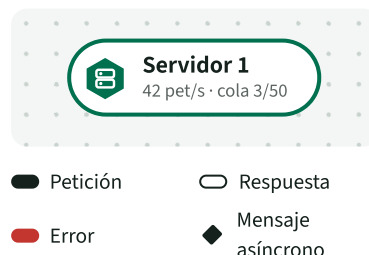
Tipografía

Source Sans 3

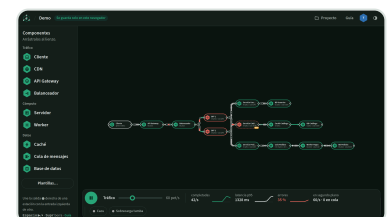
Regular · Semibold · Black —
cifras tabulares en métricas.

Humanista y legible en pantalla, cercana a la Myriad de la identidad CUJAE. Libre (OFL).

Metáfora de metro



Claro y oscuro



Mismos tokens con valores propios para el modo oscuro; contraste AA en ambos.

Iconografía · trazo de 2 px dentro del hexágono del logo



Voz

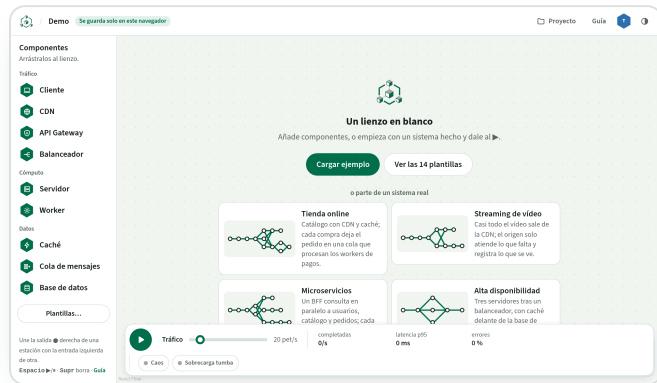
- Español claro y cercano, de tú; frases cortas.
- Se explica el porqué: «la cola crece porque llegan más de las que se procesan».
- Términos técnicos en su forma habitual (CDN, API Gateway, worker), con su ficha en la guía.

Cómo se usa

Cuatro pasos de la pantalla en blanco a un sistema funcionando. Sin instalar nada: basta un navegador.

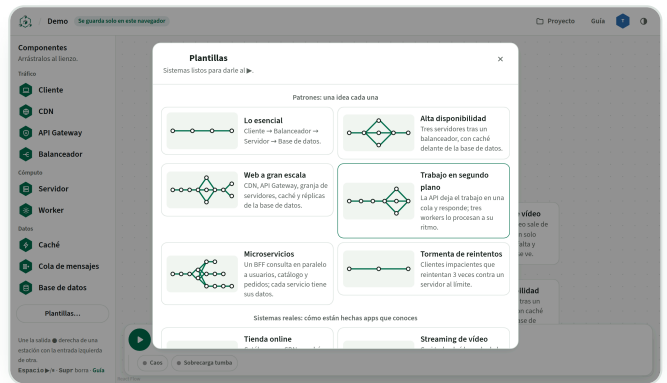
1 Entra y crea una sala

Inicia sesión con un enlace mágico o Google. «Mis salas» → «Crear sala». O prueba sin cuenta en /demo.



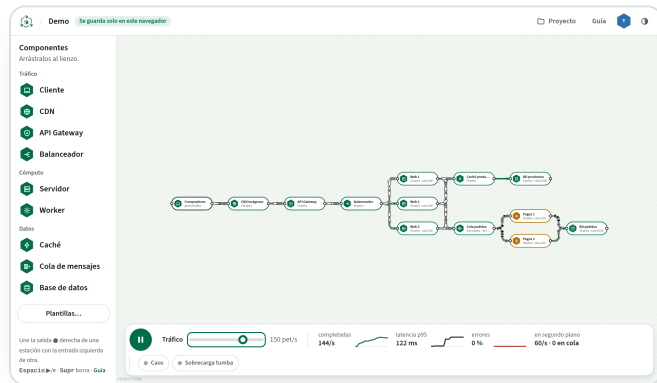
2 Diseña

Arrastra componentes desde la barra lateral o carga una de las 14 plantillas. Une la salida ● de una estación con la entrada de otra.



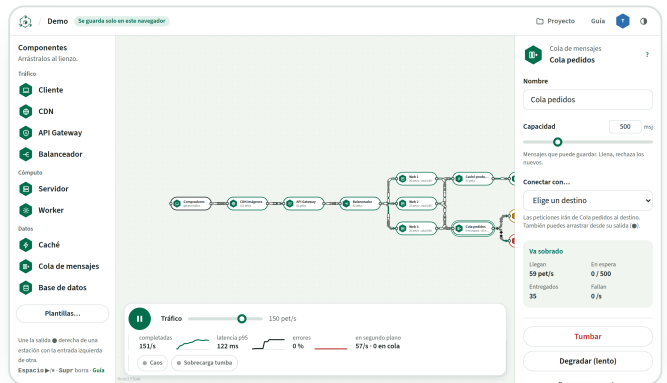
3 Pon en marcha

Pulsa ▶ (o **Espacio**) y sube el tráfico. Las estaciones se tiñen de ámbar o rojo al cargarse; las métricas se dibujan en directo.



4 Ajusta y rompe

Toca una estación: cambia sus parámetros, conéctala, tumbala o dégradala. Activa «Caos» y mira cómo se defiende el sistema.



❗ Cada componente tiene un botón ? que abre su ficha en la **Guía** integrada (/guía): qué es, cuándo usarlo, cómo se simula y sus equivalentes reales, además de retos y glosario.

Sin cuenta

Abre /demo: todo funciona y se guarda en tu navegador.

Con una sala

«Mis salas» → «Crear sala». Se guarda sola y es privada.

Uniéndote

Abre el enlace que te pasen o escribe el código de 6 caracteres.

Los 9 componentes

Cada estación es un componente real de arquitectura con un modelo de comportamiento sencillo y explicable. Todos se configuran desde el panel de propiedades.

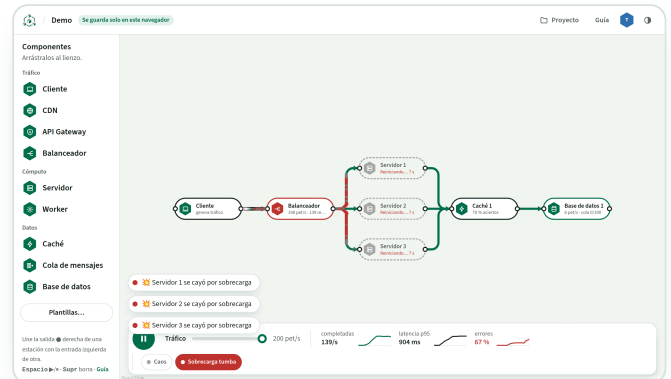
COMPONENTE	QUÉ HACE EN LA SIMULACIÓN	SE AJUSTA	EQUIVALENTES
TRÁFICO			
Cliente Representa a los usuarios (navegadores, apps móviles) que hacen peticiones al sistema.	Emite peticiones a ritmo regular. Espera la respuesta hasta su tiempo de espera; si falla o se agota, puede reintentar con espera exponencial (100 ms, 200 ms, 400 ms...).	Tiempo de espera, reintentos	Navegador web · App móvil · Otro servicio que nos llama
CDN Red de servidores repartidos por el mundo que guardan copias de imágenes, scripts y páginas.	Con probabilidad «Aciertos» responde ella misma en 2 ms; si no, pasa la petición hacia dentro.	% de aciertos	Cloudflare · Akamai · Amazon CloudFront
API Gateway Punto de entrada único de una API: autentica, enruta y limita el ritmo de las peticiones.	Cubo de fichas: se rellena a «Límite» fichas por segundo (máximo un segundo acumulado). Sin ficha, la petición se rechaza al momento.	Límite (pet/s)	Kong · NGINX · AWS API Gateway
Balanceador Reparte el tráfico entre varias copias de un servicio para sumar capacidad y tolerar caídas.	Cada «Chequeo de salud» marca como sanos los destinos encendidos y alcanzables. Entre chequeos no se entera de una caída: esas peticiones fallan. «Menos conexiones» cuenta las peticiones enviadas sin respuesta.	Round-robin / menos conexiones; chequeo de salud	HAProxy · NGINX · AWS ELB
CÓMPUTO			
Servidor Una instancia de tu aplicación: ejecuta la lógica de cada petición.	Atiende varias peticiones a la vez (capacidad × tiempo de proceso). Si están todos ocupados, la petición espera en cola; con la cola llena, falla. Si tiene destinos detrás, los llama tras procesar. En modo «a todos», espera la respuesta más lenta y falla si falla cualquiera.	Capacidad, tiempo de proceso, cola; llama a uno o a todos	Node.js · Spring Boot · Django
Worker Proceso que consume tareas de una cola: enviar correos, generar facturas, redimensionar imágenes...	Igual que un servidor. La cola solo le entrega tantos mensajes como puede procesar a la vez; si cae, los mensajes vuelven a la cola.	Capacidad, tiempo de proceso, cola	Celery · Sidekiq · Consumidores de Kafka
DATOS			
Caché Memoria muy rápida con resultados ya calculados o leídos.	Con probabilidad «Aciertos» responde en 1 ms; si no, pregunta a lo que tiene detrás.	% de aciertos	Redis · Memcached · Varnish
Cola de mensajes Intermediario asíncrono: quien envía no espera a que el trabajo se haga.	Confirma cada mensaje en cuanto lo guarda (el cliente recibe respuesta rápida) y lo entrega a sus consumidores según su capacidad libre. Si un consumidor falla, el mensaje vuelve a la cola (al menos una vez).	Capacidad (mensajes)	RabbitMQ · Apache Kafka · Amazon SQS
Base de datos Donde viven los datos de forma duradera.	Igual que un servidor: capacidad, tiempo de proceso y cola. Suele ser el cuello de botella final.	Capacidad, tiempo de proceso, cola	PostgreSQL · MySQL · MongoDB

Simulación, métricas y fallos

Un motor propio en TypeScript, determinista (paso fijo de 10 ms y semilla), separa el tiempo visual de los trenes del tiempo simulado que miden las métricas: colas, proceso y latencia de red.

Métricas en directo

Completadas/s	Respuestas correctas que llegan a los clientes.
Latencia p95	El 95 % de las peticiones tarda menos que esto (espera + proceso + red).
Errores	Rechazos, nodos caídos, líneas cortadas y tiempos de espera agotados.
Segundo plano	Mensajes procesados por los workers y mensajes en cola.
Por estación	Llegadas, rechazos, cola, carga (p) y estado, en el panel y bajo cada nombre.



«Sobrecarga tumba» activado a 200 pet/s: los tres servidores caen en cascada y se reinician.

Comportamientos que se ven

- Saturación: la cola crece, la p95 se dispara, empiezan los rechazos.
- Reintentos con espera exponencial... y tormentas de reintentos.
- Balanceadores con chequeo de salud que esquivan lo caído.
- Colas que absorben picos y reentregan mensajes fallidos.
- Fan-out: la latencia es la del servicio más lento.

Seis formas de romper un sistema

Tumbar Una estación deja de responder hasta que la revives.	Degradar Procesa 4x más lento: disco lento, vecino ruidoso, fuga de memoria.	Cortar conexión Partición de red: todo lo que se envía por esa línea falla.
Latencia de red Milisegundos extra en una línea: otra región, otro continente.	Caos Cada segundo, 15 % de probabilidad de caída, degradación o corte aleatorio.	Sobrecarga tumba Lo que pasa 5 s saturado se cae y se reinicia solo tras unos segundos.

Cómo se lee una estación

Verde Carga por debajo del 80 % y cola casi vacía.	Ámbar Carga ≥ 80 %, cola por encima del 15 % o degradada.	Rojo Carga ≥ 100 %, cola por encima del 50 % o rechazando.	Gris discontinuo Caída; muestra la cuenta atrás si se reinicia sola.
---	--	---	---

14 plantillas listas para usar

Sistemas completos que se cargan con un clic, cada uno con un experimento propuesto. Todas están verificadas por pruebas automáticas: funcionan sanas y enseñan lo que prometen.

Patrones 6 Los bloques básicos de cualquier arquitectura distribuida.



Lo esencial

Cliente → Balanceador → Servidor → Base de datos.

Prueba: Sube el tráfico a 70 pet/s; el servidor se pone rojo. Añade un segundo servidor detrás del balanceador.



Alta disponibilidad

Tres servidores tras un balanceador, con caché delante de la base de datos.

Prueba: Activa «Caos» y mira cómo el balanceador esquivo servidores caídos. ¿Qué pasa cuando cae la caché o la base de datos? Ese es un punto único de fallo.



Web a gran escala

CDN, API Gateway, granja de servidores, caché y réplicas de la base de datos.

Prueba: Pon el tráfico a 200 pet/s. Baja los aciertos de la CDN a 0 % y mira dónde aparece el cuello de botella. Luego baja el límite del gateway para protegerlo.



Trabajo en segundo plano

La API deja el trabajo en una cola y responde; tres workers lo procesan a su ritmo.

Prueba: Tumba los tres workers: la cola crece, pero los clientes siguen recibiendo respuesta. Revíuelos y mira cómo se vacía la cola.



Microservicios

Un BFF consulta en paralelo a usuarios, catálogo y pedidos; cada servicio tiene sus datos.

Prueba: La latencia es la del servicio más lento. Degrada el Servicio Catálogo y mira la p95. Tumba Usuarios: todo falla, porque el BFF necesita a los tres.



Tormenta de reintentos

Clientes impacientes que reintentan 3 veces contra un servidor al límite.

Prueba: Sube el tráfico a 55 pet/s; los reintentos multiplican la carga y el sistema colapsa. Prueba con 0 reintentos, o pon un API Gateway con límite delante.

Retos 6 Objetivos medibles para practicar en clase.

1

200 pet/s sin errores

Desde «Lo esencial»: 200 pet/s, 0 % de errores y p95 < 200 ms.

2

Sobrevive al caos

Un minuto con «Caos» activo y menos del 2 % de errores.

3

Para la tormenta

«Tormenta de reintentos» a 60 pet/s sin colapsar ni quitar reintentos.

4

El catálogo lento

«Microservicios» con el catálogo degradado y p95 < 250 ms.

5

Que los pedidos no se pierdan

Tumba la base de datos 20 s en «Trabajo en segundo plano» sin perder nada.

6

Al otro lado del océano

80 ms de latencia en la primera línea y p95 < 300 ms.

Sistemas reales 8 Arquitecturas inspiradas en productos que todos usamos.

Tienda online

Catálogo con CDN y caché; cada compra deja el pedido en una cola que procesan los workers de pagos.

Prueba: Black Friday: sube a 200 pet/s. ¿Quién se satura primero? Luego tumba un worker de pagos: los clientes no lo notan, la cola sí.

Streaming de vídeo

Casi todo el vídeo sale de la CDN; el origen solo atiende lo que falta y registra lo que se ve.

Prueba: Baja los aciertos de la CDN al 50 %: el origen, pensado para una pequeña parte del tráfico, se hunde. Así se nota cuánto trabajo hace una CDN.

Mensajería instantánea

Servidores de chat que guardan cada mensaje en una cola y consultan la presencia en una caché.

Prueba: Tumba la base de datos de mensajes 15 s: no se pierde ninguno, la cola los guarda y los reentrega al volver. Compara con quitar la cola.

Pasarela de pagos

Cada pago consulta a la vez al antifraude y a la base de transacciones; el cliente reintenta una vez.

Prueba: Degrada el servicio antifraude: todos los pagos esperan por él. Baja el tiempo de espera del cliente a 300 ms y mira cómo los pagos lentos se convierten en errores.

Red social

El feed junta en paralelo las publicaciones (con caché y réplicas) y los anuncios.

Prueba: Tumba una réplica de la base de datos: el balanceador la esquivará. Ahora baja los aciertos de la caché del timeline al 20 %: la réplica que queda no da abasto.

Sensores IoT

Miles de sensores envían lecturas a una cola de ingesta; los workers las guardan en una base de series temporales.

Prueba: Tumba la base de datos: los sensores siguen enviando y la cola lo guarda todo. Al revivirla, mira cuánto tarda en ponerse al día.

App de viajes

Pasajeros y conductores; cada viaje consulta ubicación y precio y avisa por notificación.

Prueba: Añade 150 ms de latencia a la línea hacia Precios (está en otra región): toda petición de viaje la paga. ¿Cómo lo arreglarías?

Dos regiones

Un balanceador global reparte entre Europa y América; América está a 80 ms.

Prueba: Corta la línea hacia Europa: tras el chequeo, todo va a América y sube la latencia. Si América no tiene capacidad para todo, ¿qué pasa?

Equipo, proyectos y móvil

Pensada para trabajar en clase y en grupo: lo que hace una persona lo ven todas al instante, y el trabajo sale de la plataforma en formatos útiles.



Salas colaborativas

- Enlace o código de 6 caracteres para unirse.
- Cursores y selección de cada persona, con su color.
- ▶/⏮, tráfico, caídas y fallos compartidos.
- Avisos de quién hizo qué. Guardado automático.
- Solo los miembros acceden (RLS + tokens por sala).



Menú Proyecto

Exportar imagen

PNG del diagrama completo con nombre y fecha, para trabajos y presentaciones.

Guardar

Archivo `.distinode.json` con componentes, conexiones y ajustes.

Importar

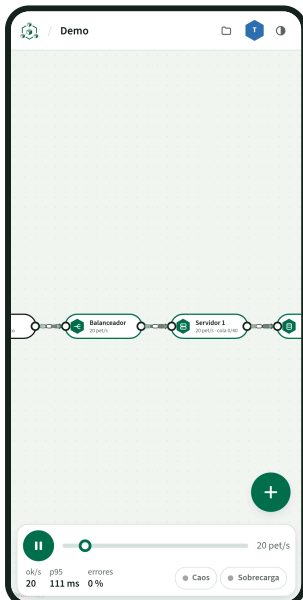
Carga un proyecto (validado) y lo comparte con toda la sala.



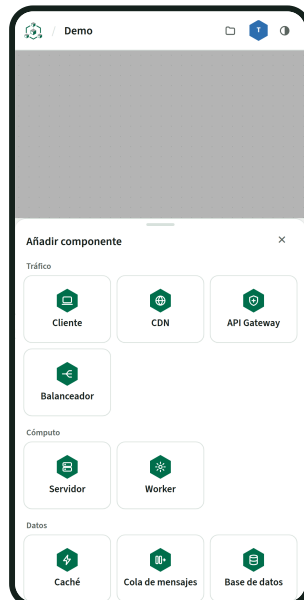
Modo demo

- Sin cuenta: `/demo` funciona en el navegador.
- Todas las funciones de diseño y simulación.
- Se guarda en el propio navegador.
- Ideal para proyectar en clase.

También en el móvil



Barra compacta con métricas



Botón + : componentes y plantillas



Propiedades en hoja inferior

- Botón + flotante para añadir componentes o cargar plantillas; se colocan solos en un hueco libre.
- Tocar una estación abre sus propiedades en una hoja inferior.
- «Conectar con...» une estaciones sin arrastrar.
- Pellizca para acercar; arrastra con un dedo para moverte.
- Puntos de conexión más grandes para el dedo.

Una clase tipo, en 45 minutos

5 min

Demostración

El profesor proyecta /demo, carga «Lo esencial» y sube el tráfico hasta saturarlo.

5 min

Equipos

Cada grupo crea una sala y comparte el código de 6 caracteres.

25 min

Retos

Cada grupo carga una plantilla y resuelve un reto, rompiendo cosas a propósito.

10 min

Entrega

Exportan el PNG para la memoria y el `.distinode.json` para corregir.

Despliegue y tecnología

El mismo producto se publica de dos maneras: en la nube, en minutos, o en un servidor propio con Docker.

RAMA PRINCIPAL

Nube · Vercel

- Vercel sirve la app Next.js.
- Supabase: autenticación (enlace mágico, Google) y Postgres con RLS.
- Liveblocks: estado compartido, presencia y avisos en tiempo real.
- Variables en `.env.example`; sin claves, la demo funciona igual.

RAMA DOCKER-POSTGRES

Servidor propio · Docker Compose

- Un solo comando: `docker compose up -d`.
- PostgreSQL + migraciones automáticas; RLS con rol sin privilegios.
- Better Auth (enlace mágico por SMTP, Google opcional).
- Servidor de tiempo real autoalojado y Caddy con HTTPS.

Pila tecnológica

Interfaz	Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS 4
Lienzo	React Flow 12 con estaciones y líneas propias; trenes en canvas a 60 fps
Tiempo real	Liveblocks: LiveMap, presencia y eventos
Datos	PostgreSQL con Row Level Security
Motor	TypeScript puro, sin dependencias, probado con Vitest

Seguridad y calidad

- Acceso a cada sala limitado a sus miembros (RLS + autorización del token de tiempo real).
- Importación de proyectos validada y acotada.
- Accesible: teclado, foco visible, ARIA, estados con texto, movimiento reducido.
- 56 pruebas automáticas del motor, plantillas y proyectos.

Referencia rápida

Espacio	Poner en marcha / pausar
Supr 	Borrar lo seleccionado
Arrastrar ●	Conectar dos estaciones
Rueda / pellizco	Acercar y alejar
Arrastrar el fondo	Mover el lienzo
? en el panel	Ficha del componente en la Guía
🕶 en la cabecera	Modo claro / oscuro



DistiNode · Diseña, simula, rompe y aprende.
Proyecto educativo · CUJAE · Sistemas Distribuidos